

0 vimrc	2	cross-product.cpp	10
vimrc	2	extend-li-chao-tree.cpp	10
1 Number Theory	2	li-chao-tree.cpp	10
Exgcd.cpp	2	max-man-dis.cpp	10
catalan.cpp	2	minimum-euc-dis.cpp	10
findPrime.cpp	2	pick-theorem.cpp	11
matrixMultiple.cpp	2	ray-casting.cpp	11
ntt.cpp	2	動態凸包.cpp	11
system-of-linear-eqn.cpp	3	多邊形重心計算.cpp	12
2 Tree	3	找三點組成最大圓.cpp	12
LCA.cpp	3	極角排序.cpp	12
MST.cpp	3	5 Data Structure	12
binary-lifing.cpp	4	Array Version of Linked List.cpp	12
find-centriod.cpp	4	Binary Index Tree.cpp	13
hld.cpp	4	Disjoint Set Union.cpp	13
small-to-large.cpp	4	Segment Tree.cpp	13
tree-diameter.cpp	4	persistence-seg-tree.cpp	13
3 Graph	5	splay-tree.cpp	14
2-sat.cpp	5	6 Not that important	14
BellmanFord.cpp	5	EPS.cpp	14
Dijkstra.cpp	5	Gauce Elimination.cpp	14
Edmond.cpp	5	Joseph.cpp	15
articulation-point.cpp	6	7 New	15
bridge.cpp	6	Convex_hull_trick.cpp	15
dinic.cpp	6	Ray_casting.cpp	15
dominator-tree.cpp	6	block_cut_tree.cpp	15
floyd-warshall.cpp	7	cross_dot_product.cpp	16
hierholzer.cpp	7	dec_string_to_bitset.cpp	16
hopcroft-karp.cpp	7	dominator_tree.cpp	16
kuhn.cpp	7	extra.cpp	17
mcmf.cpp	8	fft.cpp	17
scc.cpp	8	fwt.cpp	18
toposort.cpp	8	kmp-automaton.cpp	18
4 Geometry	8	li_chao_tree.cpp	18
Determine whether a polygon is convex.cpp	8	manacher.cpp	19
Rotating Calipers 求凸包直徑.cpp	8	shoelace.cpp	19
all solutions for Diophantine Equation in given		suffix_automaton.cpp	19
range.cpp	9	suffixarray_and_lcp.cpp	19
convex-hull.cpp	9	xor_basis.cpp	20
		zfunction_and_kmp.cpp	20

0 vimrc

vimrc

```

1 "variable
2 let mapleader = ','
3
4 "
5 set number
6 set relativenumber
7 set noexpandtab
8 set shiftwidth=4
9 set tabstop=4
10
11 inoremap kj <Esc>
12 inoremap {<CR> {<Left><CR><Esc>0
13 tnoremap kj <C-]><C-n>
14 nnoremap <leader>c :w<CR>:!g++ -std=c++20 % -o %:r &&
↪ ./%:r<CR>
15 nnoremap [ [
16 nnoremap ] ]

```

1 Number Theory

Exgcd.cpp

```

1 array<ll,2> exgcd(ll a, ll b){
2     if(b == 0) return array<ll,2>{1,0};
3     auto ans = exgcd(b, a % b);
4     return array<ll,2>{ans[1], ans[0] - a / b *
↪ ans[1]};
5 }
6

```

catalan.cpp

```

1 C_0=1
2 C_n = $\sum C_i * C_{n-1-i}$ = $\frac{1}{n+1} C_n^{2n}$

```

findPrime.cpp

```

1 constexpr ll mxN=1e6+1;
2 mobius[1]=etf[1]=1;
3 forn(i,2,mxN){
4     if(!prime[i])
↪ prime[i]=i,mobius[i]=-1,etf[i]=i-1,st[++sz]=i;
5     forn(j,1,sz+1){
6         if(i*st[j]>mxN) break;
7         prime[i*st[j]]=st[j];
8         if(i%st[j]==0){
9             mobius[i*st[j]]=0;
10            etf[i*st[j]]=etf[i]*st[j];
11            break;
12        }
13        else mobius[i*st[j]]=mobius[i]*-1,etf[i*st
↪ j]=etf[i]*(st[j]-1);
14    }
15 }

```

matrixMultiple.cpp

```

1 // mod 需外部定義; 快速冪用時記得單位矩陣
2 vector<vector<ll>> matrixMultiply(const
↪ vector<vector<ll>>& A, const vector<vector<ll>>& B) {
3     ll rowA = A.size();
4     ll colA = A[0].size();

```

```

5     ll colB = B[0].size();
6     vector<vector<ll>> C(rowA, vector<ll>(colB, 0));
7     for (ll i = 0; i < rowA; ++i) {
8         for (ll k = 0; k < colA; ++k) {
9             for (ll j = 0; j < colB; ++j) {
10                C[i][j] = (C[i][j] + A[i][k] * B[k][j]) %
↪ mod;
11            }
12        }
13    }
14    return C;
15 }

```

ntt.cpp

```

1 using ll=long long;
2 // 注意: 兩模數CRT, 結果是真值 mod 998244353*1004535809
↪ (~1.0028e18)
3 // 真實係數必須 < 該值 (值域1e9且長度>~1000就會爆),
↪ 否則需三模數NTT
4 // 長度上限 2^21 (n+m <= ~2e6): fft2 模數只有 2^21 階單位根,
↪ 超過會靜默出錯
5 class FFT{
6     public:
7         ll mod,root,root_inv,maX,M,M_inv;
8         FFT(){
9             FFT(ll a,ll b,ll c,ll d,ll e,ll
↪ f):mod(a),root(b),root_inv(c),maX(d),M
↪ (e),M_inv(f){
10                ll fsp(ll a,ll m){
11                    ll ans=1;
12                    while(m){
13                        if(m&1) ans=(ans*a)%mod;
14                        a=(a*a)%mod,m>>=1;
15                    }
16                    return ans;
17                }
18                void fft(ll sz,vc<ll> &arr,ll inv){
19                    for(ll i=1,j=0;i<sz;++i){
20                        ll bit=sz>>1;
21                        ↪ for(;j&bit;bit>>=1)
22                        ↪ j^=bit; j^=bit;
23                        if(i<j)
24                        ↪ swap(arr[i],arr[j]);
25                    }
26                    for(ll l=2;l<=sz;l<=<=1){
27                        ll wlen=inv?root_inv:root;
28                        for(ll i=l;i<maX;i<=<=1)
29                        ↪ wlen=(wlen*wlen)%mod;
30                        for(ll i=0;i<sz;i+=l){
31                            ll w=1;
32                            for(ll
↪ j=0;j<l/2;++j){
33                                ll x=arr[i
↪ +j],y=
↪ (w*arr
↪ [i+j+l
↪ /2])%m
↪ od;
34                                arr[i+j]=
↪ (x+y<mo
↪ d?x+y:
↪ x+y-mo
↪ d);
35                                arr[i+j+l/
↪ 2]=(x-
↪ y>=0?x
↪ -y:x-y
↪ +mod);
36                                w=(w*wlen)
↪ %mod;
37                            }
38                        }
39                    }
40                    if(inv){
41                        ll n_1=fsp(sz,mod-2);

```

```

38         forn(i,0,sz){
39             arr[i]=(arr[i]*n_18)
               ↳ %)mod;
40         }
41     }
42 }
43 };
44 constexpr ll MM=1002772198720536577;
45 ll mul(ll a,ll b){
46     __int128 aa=a,bb=b,cc;
47     cc=(aa*bb)%MM;
48     return (ll)cc;
49 }
50 void solve(){
51     FFT fft1(998244353,15311432,469870224,1<<23,100453589)
               ↳ 5809,332747959);
52     FFT fft2(1004535809,702606812,700146880,1<<21,99830)
               ↳ 44353,669690699);
53     ll n,m;
54     cin >> n >> m;
55     vc<ll> a(n+1),b(m+1);
56     for(auto &v:a) cin >> v;
57     for(auto &v:b) cin >> v;
58     ll sz=1;
59     for(;sz<a.size()+b.size();sz<=<=1);
60     a.resize(sz),b.resize(sz);
61     vc<ll> A=a,B=b;
62     fft1.fft(sz,a,0);
63     fft1.fft(sz,b,0);
64     forn(i,0,sz) a[i]=(a[i]*b[i])%fft1.mod;
65     fft1.fft(sz,a,1);
66
67     fft2.fft(sz,A,0);
68     fft2.fft(sz,B,0);
69     forn(i,0,sz) A[i]=(A[i]*B[i])%fft2.mod;
70     fft2.fft(sz,A,1);
71
72     forn(i,0,n+m+1) cout <<
               ↳ (mul(a[i],fft1.M*fft1.M_inv) +
               ↳ mul(A[i],fft2.M*fft2.M_inv))%MM << "
               ↳ \n"[i==n+m];
73 }
74 int main()
75 {
76     fast_io;
77     int testcase;
78     // cin>>testcase;
79     testcase=1;
80     while(testcase--)
81         solve();
82     return 0;
83 }

```

system-of-linear-eqn.cpp

```

1 // mod mdl1 下高斯消去。無解輸出-1；無窮多解「不會偵測」
   ↳ (自由變數=0給特解)，題目要區分時要自己加
2 // 輸入係數需在 [0,mdl1) 內 (負數要先轉正); inv
   ↳ 其實是快速冪，模數寫死 mdl1
3 constexpr ll mxN=505;
4 ll n,m;
5 ll inv(ll a,ll m){ll ans; for(ans=1;m;ans=(m&1?(ans*a)%mdl1
   ↳ 1:ans),a=(a*a)%mdl1,m>>=1); return
   ↳ ans;}
6 void solve(istream &cin){
7     cin>>n>>m;
8     vc<ll> res(m,0);
9     vc<vc<ll>> a(n,vc<ll>>(m+1));
10    forn(i,0,n) forn(j,0,m+1) cin>>a[i][j];
11    ll l=0;
12    for(ll i=0;i<m && l<n;++i){
13        if(!a[l][i]) forn(j,l+1,n)
               ↳ if(a[j][i]){swap(a[j],a[l]); break;}
14        if(!a[l][i]) continue;
15        ll c=inv(a[l][i],mdl1-2);
16        for(auto&v:a[l]) v=(v*c)%mdl1;

```

```

        forn(j,l+1,n){
            ll co=a[j][i];
            forn(k,i,m+1) a[j][k]=((a[j][k]-co
               ↳ *a[l][k])%mdl1+mdl1)%mdl1;
        }
        l++;
    }
    forr(i,n-1,0){
        ll j=0;
        while(j<m+1 && !a[i][j]) j++;
        if(j==a[i].size()) continue;
        if(j+1==a[i].size()){cout << -1 << '\n';
               ↳ return;}
        ll u=a[i][m];
        forn(k,j+1,m)
               ↳ u=((u-a[i][k]*res[k])%mdl1+mdl1)%mdl1;
        res[j]=u;
    }
    forn(i,0,m) cout << res[i] << " \n"[i==m-1];
}
int main()
{
    fast_io;
    ll testcase;
    // cin>>testcase;
    testcase=1;
    while(testcase--)
        solve(cin);
    return 0;
}

```

2 Tree

LCA.cpp

```

1 constexpr ll mxN=2e5+1;
2 ll pa[mxN][20],d[mxN]={};
3 vc<ll> adj[mxN];
4 void dfs(ll u=1,ll p=0){d[u]=d[p]+1,pa[u][0]=p;
   ↳ for(auto&v:adj[u]) if(v!=p) dfs(v,u);}
5 void init(ll n){ // 建好 adj 後呼叫一次
6     dfs();
7     forn(j,1,20) forn(i,1,n+1)
               ↳ pa[i][j]=pa[pa[i][j-1]][j-1];
8 }
9 ll lca(ll u,ll v){
10     if(d[u]<d[v]) swap(u,v);
11     forr(i,19,0) if(d[u]-(1<<i)>=d[v]) u=pa[u][i];
12     if(u==v) return u;
13     forr(i,19,0) if(pa[u][i]!=pa[v][i])
               ↳ u=pa[u][i],v=pa[v][i];
14     return pa[u][0];
15 }

```

MST.cpp

```

1 //kruskal
2 //vertex is 1 index
3 #define mls multiset
4
5 ll n,pa[mxN]; //memset(pa,-1,sizeof(pa));
6 mls<ar<ll,3>> adj;
7
8 ll find(ll x){return pa[x]<0?x:pa[x]=find(pa[x]);}
9 void un(ll x,ll y){
10     x=find(x),y=find(y); if(x==y) return;
11     if(-pa[x]>-pa[y]) swap(x,y); pa[y]+=pa[x],pa[x]=y;
12 }
13 ll kruskal(){ // 回傳 MST 總權重；結束後 cnt<n-1 代表圖不連通
14     ll cnt=0,wi=0;
15     while(!adj.empty() && cnt<n-1){
16         auto [w,u,v]==*adj.begin(); adj.erase(adj.begin());

```

```

17         if(find(u)!=find(v)) wi+=w,un(u,v),cnt++;
18     }
19     return wi;
20 }

```

binary-lifing.cpp

```

1  constexpr ll mxN=2e5+1;
2  ll n,q,p[mxN][31]={0};
3  vc<ll> adj[mxN];
4  void solve(istream &cin){
5      cin>>n>>q;
6      forn(i,2,n+1) cin>>p[i][0];
7      forn(j,1,31) forn(i,1,n+1) p[i][j]=p[p[i][j-1]][j-1];
8      while(q--){
9          ll x,k; cin>>x>>k;
10         k=min(k,(ll)mxN); // 跳超過深度必到0,
11         // clamp避免高位bit被忽略
12         forn(i,0,31) if(k>>i&1) x=p[x][i];
13         cout << (x?x:-1) << '\n';
14     }

```

find-centriod.cpp

```

1  void dfs(ll u=1,ll p=0){
2      d[u]=1;
3      for(auto&v:adj[u]){
4          if(v==p)
5              continue;
6          dfs(v,u);
7          d[u]+=d[v];
8      }
9  }
10 ll dfs_solve(ll u=1,ll p=0){
11     for(auto&v:adj[u]){
12         if(v==p)
13             continue;
14         if(d[v]*2>n) return dfs_solve(v,u);
15     }
16     return u;
17 }

```

hld.cpp

```

1  constexpr ll mxN=2e5+1;
2  ll tin=0,in[mxN]={},t[mxN]<<1;
3  ll n,q,a[mxN],d[mxN]={},e[mxN]={};
4  vc<ll> adj[mxN];
5  ll dfs(ll u=1,ll p=0){
6      ll mx=0,sub=1;
7      forn(i,0,adj[u].size()){
8          ll v=adj[u][i];
9          if(v!=p){d[v]=d[u]+1; ll val=dfs(v,u);
10             // sub+=val; if(val>mx) mx=val,e[u]=v;}
11             else swap(adj[u][i],adj[u][0]);
12         }
13     }
14     return sub;
15 }
16 void hld(ll u=1,ll p=1){
17     in[u]++;tin,t[in[u]+n-1]=a[u];
18     if(e[u]) hld(e[u],p);
19     forn(i,1,adj[u].size()){ll v=adj[u][i];
20         // if(e[u]!=v) hld(v,v);}
21         e[u]=p;
22     }
23 }
24 void pst(ll p,ll val){for(t[p+=n-1]=val;p>>=1;t[p]=max(t[p],
25     <<1,t[p<<1|1]));}
26 ll qry(ll l,ll r){
27     ll ans=0;

```

```

23     for(l+=n-1,r+=n-1;l<=r;l>>=1,r>>=1){if(l&1)
24         // ans=max(ans,t[l++]); if(r&1^1)
25         // ans=max(ans,t[r--]);}
26     return ans;
27 }
28 ll query(ll a,ll b){
29     ll ans=0;
30     for(;e[a]!=e[b];b=adj[e[b]][0]){
31         if(d[e[a]]>d[e[b]]) swap(a,b);
32         ans=max(ans,qry(in[e[b]],in[b]));
33     }
34     if(in[a]>in[b]) swap(a,b);
35     return max(ans,qry(in[a],in[b]));
36 }
37 void solve(istream &cin){
38     cin >> n >> q;
39     forn(i,1,n+1) cin >> a[i];
40     forn(i,0,n-1){ll u,v; cin >> u >> v;
41         // adj[u].emp(v),adj[v].emp(u);}
42         adj[u].emp(0);
43         dfs();
44         hld();
45         forr(i,n-1,1) t[i]=max(t[i<<1],t[i<<1|1]);
46         for(ll op,a,b;q--){
47             cin >> op >> a >> b;
48             if(op==1) pst(in[a],b);
49             else cout << query(a,b) << ' ';
50         }
51     }

```

small-to-large.cpp

```

1  // 注意: 每層遞迴持有 deque, 鏈狀樹會爆棧且記憶體~300MB,
2  // 深樹題慎用
3  constexpr ll mxN=2e5+1;
4  ll n,k,res=0;
5  vc<ll> adj[mxN];
6  deque<ll> dfs(ll u=1,ll p=0){
7      deque<ll> cur(1,{1}),tmp;
8      for(auto &v:adj[u]){
9          if(v!=p){
10             tmp=dfs(v,u),tmp.push_front(0);
11             if(tmp.size()>cur.size())
12                 swap(tmp,cur);
13             forn(i,0,tmp.size()) if(k-i>=0 &&
14                 // k-i<cur.size())
15                 // res+=cur[k-i]*tmp[i];
16                 forn(i,0,tmp.size())
17                     // cur[i]+=tmp[i];
18             }
19         }
20     }
21     return cur;
22 }
23 void solve(istream &cin){
24     cin >> n >> k;
25     forn(i,0,n-1){ll u,v; cin >> u >> v;
26         // adj[u].emp(v),adj[v].emp(u);}
27         dfs();
28         cout << res << '\n';
29     }

```

tree-diameter.cpp

```

1  #define emp emplace_back
2  constexpr ll mxN=2e5+1;
3  ll n,res=0;
4  vc<ll> adj[mxN],dp(mxN);
5
6  void dfs(ll u=1,ll p=0){dp[u]=0; for(auto&v:adj[u])
7      // if(v!=p) dfs(v,u),res=max(res,dp[u]+dp[v]+1),dp[u]=max
8      // (dp[u],dp[v]+1);}
9
10 int main()
11 {

```

```

10     cin>>n; forn(i,0,n-1){ll u,v; cin>>u>>v;
    ↪ adj[u].emp(v),adj[v].emp(u);}
11     dfs();
12     cout << res << '\n';
13 }

```

3 Graph

2-sat.cpp

```

1  constexpr ll mxN=2e5+1;
2  ll sc=0,scc[mxN];
3  ll id=0,st[mxN],vs[mxN]{};
4  ll n,m,tin=0,dfn[mxN]{};low[mxN];
5  vc<ll> adj[mxN];
6  void tarjan(ll u){
7      dfn[u]=low[u]=++tin,st[++id]=u,vs[u]=1;
8      for(auto &v:adj[u]){
9          if(!dfn[v])
    ↪ tarjan(v),low[u]=min(low[u],low[v]);
    else if(vs[v]) low[u]=min(low[u],dfn[v]);
10     }
11     if(low[u]==dfn[u]){
12         sc++;
13         for(;id;){
14             scc[st[id]]=sc,vs[st[id]]=0;
15             id--;
16             if(st[id+1]==u) break;
17         }
18     }
19 }
20 void solve(istream &cin){
21     cin >> n >> m;
22     forn(i,1,n+1){
23         char c1,c2; ll x,y,X,Y; cin >> c1 >> x >>
    ↪ c2 >> y;
24         X=x+m,Y=y+m;
25         if(c1=='-') swap(x,X);
26         if(c2=='-') swap(y,Y);
27         adj[X].emp(y),adj[Y].emp(x);
28     }
29     forn(i,1,2*m+1) if(!dfn[i]) tarjan(i);
30     forn(i,1,m+1) if(scc[i]==scc[i+m]){cout <<
    ↪ "IMPOSSIBLE" << '\n'; return;}
31     forn(i,1,m+1) cout << (scc[i]>scc[i+m]?'-':'+') <<
    ↪ " \n"[i==m];
32 }

```

BellmanFord.cpp

```

1  //SPFA; 回傳false=有負環(僅偵測從start可達的);
    ↪ d[]需外部初始化INF, d[start]=0
2  bool spfa(){
3      vc<ll> vs(n+1,0); //1-indexed 要開 n+1
4      queue<ll> q; q.emplace(start);
5      while(!q.empty()){
6          ll u=q.front(); q.pop(),vs[u]=0;
7          for(auto&[v,w]:adj[u]){
8              if(d[v]>d[u]+w){
9                  d[v]=d[u]+w;
10                 if(!vs[v]){vs[v]=1,cnt[v]++; if(cnt[v]>n)
    ↪ return false; q.emplace(v);}
11             }
12         }
13     }
14     return true;
15 }

```

Dijkstra.cpp

```

1  void djc(){
2      mls<pll> hp;
3      res[1]=0,hp.emplace(pll{res[1],1});
4      while(!hp.empty()){
5          auto [d,u]=*hp.bg(); hp.erase(hp.bg());
6          if(d>res[u]) continue;
7          for(auto&[v,w]:adj[u]){
8              if(w+res[u]<res[v]) res[v]=w+res[u]
    ↪ ,hp.emplace(pll{res[v],v});
9          }
10     }
11 }

```

Edmond.cpp

```

1  constexpr ll mxN=505;
2  ll n,m,adj[mxN][mxN],g[mxN][mxN]{}; //adj=capacity
3  vc<ll> pa;
4  ll bfs(){
5      queue<pll> q; q.emplace(pll{0,inf});
6      while(!q.empty()){
7          auto [u,flow]=q.front(); q.pop();
8          forn(i,0,n)
    ↪ if(g[u][i] && adj[u][i] &&
    ↪ pa[i]==-1){
9              pa[i]=u;
10             if(i==n-1) return
    ↪ min(flow,adj[u][i]);
11             q.emplace(pll{i,min(flow,a
    ↪ dj[u][i])});
12         }
13     }
14     return 0;
15 }
16 void solve(istream &cin){
17     cin>>n>>m;
18     pa.resize(n);
19     forn(i,1,m+1){ll u,v; cin>>u>>v,u--,v--;
    ↪ adj[u][v]++,g[u][v]++,adj[v][u]++,g[v][u]++;}
20     ↪ //重邊容量/重數要累加
21     ll f=0;
22     fill(all(pa),-1),pa[0]=0;
23     for(ll nf;nf=bfs();f+=nf){
24         ll cur=n-1;
25         while(cur) adj[pa[cur]][cur]-=nf,adj[cur][
    ↪ pa[cur]]+=nf,cur=pa[cur];
26         fill(all(pa),-1),pa[0]=0;
27     }
28     vc<pll> res;
29     forn(i,0,n) forn(j,0,n) if(g[i][j] && pa[i]!=-1 &&
    ↪ pa[j]==-1) forn(_,0,g[i][j])
    ↪ res.emplace(pll{i,j}); //重邊每條都要列
30     cout << res.size() << '\n';
31     for(auto&[u,v]:res) cout << u+1 << ' ' << v+1 <<
    ↪ '\n';
32 }
33 int main()
34 {
35     fast_io;
36     ll testcase;
37     // cin>>testcase;
38     testcase=1;
39     while(testcase--){
40         solve(cin);
41         return 0;
42     }
43 }

```

articulation-point.cpp

```

1 //undirected graph articulation point; root呼叫
2 ↪ tarjan(1,0), 不連通圖每個分量都要跑
3 ll n,tim=0,art=0;
4 vc<ll> dfn(mxN,0),low(mxN);
5 void tarjan(ll u,ll p){
6     ll cnt=0; bool isArt=false;
7     dfn[u]=low[u]=++tim;
8     for(auto&v:adj[u]){
9         if(!dfn[v]){
10             tarjan(v,u);
11             low[u]=min(low[u],low[v]);
12             if(low[v]>=dfn[u] && p) isArt=true; //非root:
13             ↪ 某子樹回不到u之上
14             cnt++;
15         }
16         else if(v!=p) low[u]=min(low[u],dfn[v]);
17     }
18     if(!p && cnt>1) isArt=true; //root: 至少兩個子樹
19     art+=isArt;
20 }

```

bridge.cpp

```

1 constexpr ll mxN=1e5+1;
2 ll n,m,dfn[mxN]={},low[mxN],tin=0;
3 vc<ll> adj[mxN];
4 vc<ar<ll,2>> res;
5 void tarjan(ll u=1,ll p=0){
6     dfn[u]=low[u]=++tin;
7     bool skip=0; //父邊只跳過一次,
8     ↪ 重邊(平行邊)才不會被誤判成橋
9     for(auto &v:adj[u]){
10         if(v==p && !skip){skip=1; continue;}
11         if(dfn[v]) low[u]=min(low[u],dfn[v]);
12         else{
13             tarjan(v,u),low[u]=min(low[u],low[v]);
14             ↪ v];
15             if(low[v]>dfn[u])
16                 ↪ res.emp(ar<ll,2>{u,v});
17         }
18     }
19 }
20 }

```

dinic.cpp

```

1
2 constexpr ll mxN=505;
3 class fe{
4     public:
5         ll u,v,cp,fw;
6         fe(){
7             fe(ll U,ll V,ll CP):u(U),v(V),cp(CP){fw=0;}
8         };
9         ll n,m,sz=0;
10         vc<fe> edg;
11         queue<ll> q;
12         vc<ll> adj[mxN],ptr,dep;
13         bool bfs(){
14             while(!q.empty()){
15                 ll u=q.front(); q.pop();
16                 for(auto&id:adj[u]){
17                     if(edg[id].cp==edg[id].fw ||
18                         ↪ dep[edg[id].v]!=-1) continue;
19                     dep[edg[id].v]=dep[u]+1,q.emplace(
20                         ↪ edg[id].v);
21                 }
22             }
23             return dep[n-1]!=-1;
24         }
25         ll dinic(ll u,ll push){
26             if(!push) return 0;
27         }
28     }
29 }

```

```

30         if(u==n-1) return push;
31         for(ll &cid=ptr[u];cid<adj[u].size();++cid){
32             ll id=adj[u][cid];
33             ll v=edg[id].v;
34             if(dep[v]!=dep[u]+1) continue;
35             ll tr=dinic(v,min(push,edg[id].cp-edg[id].
36                 ↪ fw));
37             if(!tr) continue;
38             edg[id].fw+=tr,edg[id^1].fw-=tr;
39             return tr;
40         }
41         return 0;
42     }
43     void solve(istream &cin){
44         cin>>n>m;
45         ptr.resize(n),dep.resize(n);
46         forn(i,1,m+1){
47             ll u,v,c; cin>>u>>v>>c,u--,v--;
48             edg.emp(fe{u,v,c}),edg.emp(fe{v,u,0});
49             adj[u].emp(sz),adj[v].emp(sz+1),sz+=2;
50         }
51         ll res=0;
52         for(;;){
53             ll flow;
54             fill(all(dep),-1);
55             q.emplace(0),dep[0]=0;
56             if(!bfs()) break;
57             fill(all(ptr),0);
58             for(;;flow=dinic(0,inf);res+=flow);
59         }
60         cout << res << '\n';
61     }
62 }
63 int main()
64 {
65     fast_io;
66     ll testcase;
67     // cin>>testcase;
68     testcase=1;
69     while(testcase--){
70         solve(cin);
71         return 0;
72     }
73 }

```

dominator-tree.cpp

```

1 constexpr ll mxN=1e5+1;
2 ll n,m,tim=0,id[mxN],sd[mxN],pa[mxN],f[mxN],lb[mxN],et[mxN]
3 ↪ []{},ret[mxN];
4 vc<ll> adj[mxN],radj[mxN],t[mxN],st;
5 void dfs(ll u=1){
6     et[u]=++tim,ret[tim]=u;
7     id[tim]=sd[tim]=lb[tim]=pa[tim]=tim;
8     for(auto&v:adj[u]){
9         if(!et[v]) dfs(v),f[et[v]]=et[u];
10         radj[et[v]].emp(et[u]);
11     }
12 }
13 ll find(ll u,ll x=0){
14     if(u==pa[u]) return x?-1:u;
15     ll v=find(pa[u],x+1);
16     if(v<0) return u;
17     if(sd[lb[u]]>sd[lb[pa[u]]]) lb[u]=lb[pa[u]];
18     pa[u]=v;
19     return x?v:lb[u];
20 }
21 void built(){
22     vc<ll> arr[n+1];
23     forr(i,n,1){
24         for(auto&v:radj[i])
25             ↪ sd[i]=min(sd[i],sd[find(v)]);
26         if(i>1) arr[sd[i]].emp(i);
27         for(auto&v:arr[i]){
28             ll x=find(v);
29             if(sd[x]==sd[v]) id[v]=sd[v];
30             else id[v]=x;
31         }
32     }
33 }

```

```

30         if(i>1) pa[i]=f[i];
31     }
32     forn(i,2,n+1){
33         if(id[i]!=sd[i]) id[i]=id[id[i]];
34         t[ret[i]].emp(ret[id[i]]),t[ret[id[i]]].emp
            ↪ p(ret[i]);
35     }
36 }
37 void dfs_ans(ll u=1,ll p=0){
38     st.emp(u);
39     if(u==n){
40         sort(all(st));
41         cout << st.size() << '\n';
42         for(auto&v:st) cout << v << ' '; cout <<
            ↪ '\n';
43         exit(0);
44     }
45     for(auto&v:t[u]){
46         if(v==p) continue;
47         dfs_ans(v,u);
48     }
49     st.pop_back();
50 }
51 void solve(istream &cin){
52     cin>>n>>m;
53     forn(i,1,m+1){
54         ll u,v; cin>>u>>v;
55         adj[u].emp(v);
56     }
57     dfs();
58     built();
59     dfs_ans();
60 }
61

```

floyd-warshall.cpp

```

1 // 先設 f[x][x]=0 其餘INF
   ↪ (INF用0x3f3f3f3f3f3f3f3f可安全相加)
2 for (ll k = 1; k <= n; k++) {
3     for (ll x = 1; x <= n; x++) {
4         for (ll y = 1; y <= n; y++) {
5             f[x][y] = min(f[x][y], f[x][k] + f[k][y]);
6         }
7     }
8 }

```

hierholzer.cpp

```

1 //debrujin sequence
2 ll n;
3 string s;
4 vc<ll> adj[1<<14|1];
5 void dfs(ll u=0){
6     while(!adj[u].empty()){
7         ll v; v=adj[u].back(); adj[u].pop_back();
8         dfs(v);
9         s+=(v&1)?'1':'0';
10    }
11 }
12 void solve(istream &cin){
13     cin>>n;
14     if(n==1){cout << "01" << '\n'; return;}
15     forn(i,0,1<<(n-1)){
16         adj[i].emp((i<<1)&((1<<(n-1))-1));
17         adj[i].emp((i<<1|1)&((1<<(n-1))-1));
18     }
19     dfs();
20     forn(i,0,n-1) s+='0';
21     reverse(all(s));
22     cout << s << '\n';
23 }

```

hopcroft-karp.cpp

```

1 constexpr ll mxN=501;
2 ll n,m,k,mat[mxN<<1]{},d[mxN<<1];
3 vc<ll> adj[mxN];
4 ll bfs(){
5     queue<ll> q;
6     memset(d,-1,sizeof(d));
7     forn(i,1,n+1) if(!mat[i]) d[i]=0,q.emplace(i);
8     while(!q.empty()){
9         ll u=q.front(); q.pop();
10        if(~d[0]) return 1;
11        for(auto &v:adj[u]){
12            if(!~d[mat[v]]) d[mat[v]]=d[u]+1
            ↪ ,q.emplace(mat[v]);
13        }
14    }
15    return 0;
16 }
17 bool dfs(ll u){
18     if(!u) return true;
19     for(auto &v:adj[u]) if(d[mat[v]]==d[u]+1 &&
        ↪ dfs(mat[v])){mat[u]=v,mat[v]=u;return true;}
20     d[u]=-1;
21     return false;
22 }
23 void solve(istream &cin){
24     cin >> n >> m >> k;
25     forn(i,0,k){ll u,v; cin >> u >> v;
        ↪ adj[u].emp(n+v);}
26     ll res=0;
27     for(;bfs();){forn(i,1,n+1) if(!mat[i]) res+=dfs(i);
28     cout << res << '\n';
29     if(res) forn(i,1,n+1) if(mat[i]) cout << i << ' '
        ↪ << (mat[i]-n) << '\n';
30 }

```

kuhn.cpp

```

1 constexpr ll mxN=1001;
2 constexpr ll base=500;
3 ll n,m,k,vs1[mxN],vs2[mxN],ok[mxN][mxN]{};
4 vc<ll> adj[mxN];
5 bool dfs(ll u){
6     if(vs2[u]) return false;
7     vs2[u]=1;
8     for(auto&v:adj[u]) if(!vs1[v] ||
        ↪ dfs(vs1[v])){vs1[v]=u; return true;}
9     return false;
10 }
11 void solve(istream &cin){
12     cin >> n >> m >> k;
13     //[1,500] boy [500+1,500+500] girl
14     forn(i,0,k){
15         ll u,v; cin >> u >> v,v+=base;
16         if(!ok[u][v])
17             adj[u].emp(v),adj[v].emp(u),ok[u][
            ↪ v]=1;
18     }
19     memset(vs1,0,sizeof(vs1));
20     forn(i,1,n+m+1){
21         memset(vs2,0,sizeof(vs2));
22         dfs(i);
23     }
24     vc<ar<ll,2>> res;
25     forn(i,1,n+1){
26         for(auto &v:adj[i]){
27             if(vs1[v]==i)
                ↪ res.emp(ar<ll,2>{i,v-base});
28         }
29     }
30     cout << res.size() << '\n';
31     for(auto&[u,v]:res) cout << u << ' ' << v << '\n';
32 }

```

mcmf.cpp

```

1  class Edge{
2      public:
3          ll u,v,c,w;
4          Edge(){
5              Edge(ll u,ll v,ll c,ll
6                  ↪ w):u(u),v(v),c(c),w(w){}
7  };
8  constexpr ll mxN=501;
9  ll n,m,k,id=0,vs[mxN],d[mxN],pa[mxN];
10 vc<Edge> e;
11 vc<ll> adj[mxN];
12 ll bfs(){
13     memset(pa,-1,8*(n+1)),memset(d,0x3f,sizeof(d));
14     queue<ll> q;
15     d[1]=0,vs[1]=1,q.emplace(1);
16     while(!q.empty()){
17         auto u=q.front(); q.pop(),vs[u]=0;
18         for(auto &eid:adj[u]){
19             auto [x,v,c,w]=e[eid];
20             if(c && d[u]+w<d[v]){
21                 pa[v]=eid,d[v]=d[u]+w;
22                 if(!vs[v])
23                     ↪ vs[v]=1,q.emplace(v);
24             }
25         }
26     }
27     return d[n]<INF;
28 }
29 void solve(){
30     cin >> n >> m >> k;
31     forn(i,1,m+1){
32         ll u,v,c,w; cin >> u >> v >> c >> w;
33         e.emp(Edge{u,v,c,w}),e.emp(Edge{v,u,0,-w});
34         ↪ ,adj[u].emp(id++),adj[v].emp(id++);
35     }
36     ll f=0,res=0;
37     for(ll nf,v;f<k && bfs();){
38         for(nf=k-f,v=n;v!=1;nf=min(nf,e[pa[v]].c),
39             ↪ v=e[pa[v]].u);
40         for(v=n;v!=1;e[pa[v]].c-=nf,e[pa[v]^1].c+=nf,
41             ↪ nf,v=e[pa[v]].u);
42         res+=d[n]*nf,f+=nf;
43     }
44     cout << (f<k?-1:res) << '\n';
45 }

```

scc.cpp

```

1  void dfs(ll u){
2      dfn[u]=low[u]=++tim,st[++id]=u,vs[u]=1;
3      for(auto &v:adj[u]){
4          if(!dfn[v])
5              ↪ dfs(v),low[u]=min(low[u],low[v]);
6          else if(vs[v]) low[u]=min(low[u],dfn[v]);
7      }
8      if(low[u]==dfn[u]){
9          for(ll v=-1;v!=u;scc[st[id]]=sc,vs[st[id]]
10             ↪ =0,v=st[id],id--);
11          sc++;
12      }
13 }

```

toposort.cpp

```

1  void topo_sort() {
2      int n, m;
3      cin >> n >> m;
4      vector<vector<int>> graph(n+1);
5      for (int i = 0; i < m; i++) {
6          int a, b;
7          cin >> a >> b;

```

```

8          graph[a].push_back(b);
9      }
10     vector<int> rdu(n+1, 0);
11     for (auto &node : graph) {
12         for (auto &now : node) {
13             rdu[now]++;
14         }
15     }
16     queue<int> que;
17     for (int i = 1; i <= n; i++) {
18         if (rdu[i] == 0) que.push(i);
19     }
20     vector<int> topo;
21     while (!que.empty()) {
22         int top = que.front();
23         que.pop();
24         topo.push_back(top);
25         for (auto &i : graph[top]) {
26             rdu[i]--;
27             if (rdu[i] == 0) que.push(i);
28         }
29     }
30     if (topo.size() == n) {
31         for (auto &i : topo) cout << i << " ";
32     }
33     else
34         cout << "IMPOSSIBLE";
35 }

```

4 Geometry

Determine whether a polygon is convex.cpp

```

1  #include <vector>
2  using namespace std;
3
4  struct Point {
5      double x, y;
6  };
7
8  double cross(const Point& a, const Point& b, const Point&
9  ↪ c) {
10     // 計算向量 ab x bc
11     double x1 = b.x - a.x, y1 = b.y - a.y;
12     double x2 = c.x - b.x, y2 = c.y - b.y;
13     return x1 * y2 - y1 * x2;
14 }
15
16 bool isConvex(const vector<Point>& poly) {
17     int n = poly.size();
18     if (n < 3) return false;
19     int sign = 0;
20     for (int i = 0; i < n; ++i) {
21         double c = cross(poly[i], poly[(i + 1) % n],
22             ↪ poly[(i + 2) % n]);
23         if (c != 0) {
24             if (sign == 0)
25                 sign = (c > 0) ? 1 : -1;
26             else if ((c > 0 && sign < 0) || (c < 0 && sign
27                 ↪ > 0))
28                 return false;
29         }
30     }
31     return true;
32 }

```

Rotating Calipers 求凸包直徑.cpp

```

1  struct Point {
2      double x, y;
3  };
4

```



```

5 double dist2(const Point& a, const Point& b) {
6     double dx = a.x - b.x, dy = a.y - b.y;
7     return dx * dx + dy * dy;
8 }
9
10 // 假設 convex 是已經逆時針排序的凸包
11 double convexHullDiameter(const vector<Point>& convex) {
12     int n = convex.size();
13     if (n == 1) return 0;
14     if (n == 2) return dist2(convex[0], convex[1]);
15     double res = 0;
16     for (int i = 0, j = 1; i < n; ++i) {
17         while (dist2(convex[i], convex[(j + 1) % n]) >
18             dist2(convex[i], convex[j]))
19             j = (j + 1) % n;
20         res = max(res, dist2(convex[i], convex[j]));
21     }
22     return sqrt(res);
23 }

```

all solutions for Diophantine Equation in given range.cpp

```

1 /*
2     1. General solution to the linear Diophantine equation:
3      $a * x + b * y = c$ 
4     If  $(x_0, y_0)$  is a particular solution and  $g = \gcd(a, b)$ :
5      $x = x_0 + k * (b / g)$ 
6      $y = y_0 - k * (a / g)$ 
7     for any integer  $k$ 
8     2. Number of integer solutions  $(x, y)$  within bounds:
9      $minx \leq x \leq maxx$ 
10     $miny \leq y \leq maxy$ 
11    If  $lx$  and  $rx$  are the valid range for  $x$ :
12    Number of solutions =  $(rx - lx) / \gcd(a, b) + 1$ 
13 */
14 void shift_solution(int & x, int & y, int a, int b, int
15     cnt) {
16     x += cnt * b;
17     y -= cnt * a;
18 }
19 int find_all_solutions(int a, int b, int c, int minx, int
20     maxx, int miny, int maxy) {
21     int x, y, g;
22     if (!find_any_solution(a, b, c, x, y, g))
23         return 0;
24     a /= g;
25     b /= g;
26     int sign_a = a > 0 ? +1 : -1;
27     int sign_b = b > 0 ? +1 : -1;
28     shift_solution(x, y, a, b, (minx - x) / b);
29     if (x < minx)
30         shift_solution(x, y, a, b, sign_b);
31     if (x > maxx)
32         return 0;
33     int lx1 = x;
34     shift_solution(x, y, a, b, (maxx - x) / b);
35     if (x > maxx)
36         shift_solution(x, y, a, b, -sign_b);
37     int rx1 = x;
38     shift_solution(x, y, a, b, -(miny - y) / a);
39     if (y < miny)
40         shift_solution(x, y, a, b, -sign_a);
41     if (y > maxy)
42         return 0;
43     int lx2 = x;
44     shift_solution(x, y, a, b, -(maxy - y) / a);
45     if (y > maxy)
46         shift_solution(x, y, a, b, sign_a);
47 }

```

```

51 int rx2 = x;
52
53 if (lx2 > rx2)
54     swap(lx2, rx2);
55 int lx = max(lx1, lx2);
56 int rx = min(rx1, rx2);
57
58 if (lx > rx)
59     return 0;
60 return (rx - lx) / abs(b) + 1;
61 }

```

convex-hull1.cpp

```

1 class Point{
2     public:
3         ll x,y;
4         Point(){}
5         Point(ll x,ll y):x(x),y(y){}
6         friend ll operator^(const Point a,const
7             Point b){return a.x*b.y-a.y*b.x;}
8         friend ll operator*(const Point a,const
9             Point b){return a.x*b.x+a.y*b.y;}
10        friend Point operator-(const Point a,const
11            Point b){
12            return Point{a.x-b.x,a.y-b.y};
13        }
14        friend ostream& operator>>(ostream
15            &in,Point &a){
16            in >> a.x >> a.y;
17            return in;
18        }
19        friend ostream& operator<<(ostream
20            &out,Point &a){
21            out << a.x << ' ' << a.y << '\n';
22            return out;
23        }
24        friend bool operator<(const Point a,const
25            Point b){return a.x<b.x|| (a.x==b.x &&
26                a.y<b.y);}
27 };
28 ll dis(Point a,Point b){
29     return (a.x-b.x)*(a.x-b.x)+(a.y-b.y)*(a.y-b.y);
30 }
31 ll ok(Point a,Point b,Point c){
32     ll ang=(b-a)^(c-a); //ang>0 false ang< true
33     return ang<0;
34 }
35 constexpr ll mxN=2e5+5;
36 ll st[mxN],id=0;
37 void solve(istream &cin){
38     ll n; cin >> n;
39     vc<Point> a(n); for(auto &v:a) cin >> v;
40     forn(i,1,n) if(a[i]<a[0]) swap(a[i],a[0]);
41     sort(1+all(a),[p=a[0]](Point a,Point b){ll
42         ang=(a-p)^(b-p); return
43         !ang?dis(p,a)<dis(p,b):ang>0;});
44     {
45         ll j; for(j=n-1;j>=0 &&
46             ((a[j]-a[0])^(a[n-1]-a[0]))==0;j--);
47         reverse(j+1+a.bg(),a.ed());
48     }
49     forn(i,0,n){
50         while(id>1 &&
51             ok(a[st[id-1]],a[st[id]],a[i])) --id;
52         st[++id]=i;
53     }
54     cout << id << '\n';
55     if(id) for(;id;cout << a[st[id--]]);
56 }

```

cross-product.cpp

```

1 class Point{
2     public:
3         ll x,y;
4         Point(){}
5         Point(ll x,ll y):x(x),y(y){}
6         ll f(Point a,Point b){
7             return a.x*b.y-b.x*a.y;
8         }//cross product
9         friend Point operator-(const Point a,const
10             ↪ Point b){
11             return Point{a.x-b.x,a.y-b.y};
12         }
13         friend istream& operator>>(istream
14             ↪ &in,Point &a){
15             in >> a.x >> a.y;
16             return in;
17         }
18     };

```

extend-li-chao-tree.cpp

```

1 class Line{
2     public:
3         ll m,c;
4         Line(){m=0,c=-1;}
5         Line(ll m,ll c):m(m),c(c){}
6         ll cal(ll x){return m*x+c;}
7     };
8     constexpr ll mxN=1e5;
9     Line t[mxN<<2];
10    void insert(ll i,ll sl,ll sr,ll l,ll r,Line L){
11        if(l>sr || r<sl) return ;
12        ll mid=(sl+sr)>>1;
13        if(l<=sl && sr<=r){
14            if(L.cal(mid)>t[i].cal(mid)) swap(L,t[i]);
15            if(sl==sr) return;
16            if(L.m>t[i].m)
17                ↪ insert(i<<1|1,mid+1,sr,l,r,L);
18            else insert(i<<1,sl,mid,l,r,L);
19            return ;
20        }
21        insert(i<<1,sl,mid,l,r,L);
22        insert(i<<1|1,mid+1,sr,l,r,L);
23    }
24    ll query(ll i,ll sl,ll sr,ll x){
25        ll mid=(sl+sr)>>1;
26        if(sl==sr) return t[i].cal(x);
27        if(x<=mid) return
28            ↪ max(t[i].cal(x),query(i<<1,sl,mid,x));
29        else return
30            ↪ max(t[i].cal(x),query(i<<1|1,mid+1,sr,x));
31    }
32    void solve(istream &cin){
33        ll n,m,r0=-1; cin >> n >> m;
34        forn(i,0,n){
35            ll x1,x2,y1,y2; cin >> x1 >> y1 >> x2 >>
36            ↪ y2;
37            if(!x1) r0=max(r0,y1);
38            Line l((y2-y1)/(x2-x1),-x1*(y2-y1)/(x2-x1)+
39            ↪ y1);
40            insert(1,1,mxN,x1,x2,1);
41        }
42        cout << r0 << " \n"[m==1];
43        forn(i,1,m+1) cout << query(1,1,mxN,i) << "
44            ↪ \n"[i==m];
45    }

```

li-chao-tree.cpp

```

1 class Line{
2     public:
3         ll m,c;

```

```

        Line(){m=c=0;}
        Line(ll m,ll c):m(m),c(c){}
        ll cal(ll x){return m*x+c;}
    };
    constexpr ll mxN=1e5;
    Line t[mxN<<2];
    void insert(ll i,ll sl,ll sr,Line l){
        ll mid=(sl+sr)>>1;
        if(l.cal(mid)>t[i].cal(mid)) swap(l,t[i]);
        if(sl==sr) return;
        if(l.m>t[i].m) insert(i<<1|1,mid+1,sr,l);
        else insert(i<<1,sl,mid,l);
    }
    ll query(ll i,ll sl,ll sr,ll x){
        ll mid=(sl+sr)>>1;
        if(sl==sr) return t[i].cal(x);
        if(x<=mid) return
        ↪ max(t[i].cal(x),query(i<<1,sl,mid,x));
        else return
        ↪ max(t[i].cal(x),query(i<<1|1,mid+1,sr,x));
    }
    void solve(istream &cin){
        ll n,m; cin >> n >> m;
        forn(i,0,n){
            ll y1,y2; cin >> y1 >> y2;
            // (y2-y1)/(m-0) = (y-y1)/x
            //x * (y2-y1)/m + y1= y
            Line l((y2-y1)/m,y1);
            insert(1,1,mxN,l);
        }
        forn(i,0,m+1) cout << query(1,1,mxN,i) << "
        ↪ \n"[i==m];
    }

```

max-man-dis.cpp

```

1 constexpr ll mxN=2e5+1;
2 ll n,d[2][2];
3 void solve(){
4     cin >> n;
5     d[0][0]=d[1][0]=-inf;
6     d[0][1]=d[1][1]=inf;
7     forn(i,0,n){
8         ll x,y;
9         cin >> x >> y;
10        d[0][0]=max(d[0][0],x+y);
11        d[0][1]=min(d[0][1],x+y);
12        d[1][0]=max(d[1][0],x-y);
13        d[1][1]=min(d[1][1],x-y);
14        cout <<
15            ↪ max(d[0][0]-d[0][1],d[1][0]-d[1][1])
16            ↪ << '\n';
17    }

```

minimum-euc-dis.cpp

```

1 class Point{
2     public:
3         ll x,y;
4         Point(){}
5         Point(ll x,ll y):x(x),y(y){}
6         friend ll operator^(const Point a,const
7             ↪ Point b){return a.x*b.y-a.y*b.x;}
8         friend ll operator*(const Point a,const
9             ↪ Point b){return a.x*b.x+a.y*b.y;}
10        friend Point operator-(const Point a,const
11            ↪ Point b){
12            return Point{a.x-b.x,a.y-b.y};
13        }
14        friend istream& operator>>(istream
15            ↪ &in,Point &a){
16            in >> a.x >> a.y;

```

```

13         return in;
14     }
15     friend bool operator<(const Point a, const
    ↪ Point b){return a.x<b.x || (a.x==b.x &&
    ↪ a.y<b.y);}
16 };
17 ll euc(Point a, Point b){
18     return (a.x-b.x)*(a.x-b.x)+(a.y-b.y)*(a.y-b.y);
19 }
20 void solve(istream &cin){
21     ll n; cin >> n;
22     vc<Point> a(n); for(auto &v:a) cin >> v;
23     sort(all(a), [](Point u, Point v){return
    ↪ (u.y<v.y) || (u.y==v.y && u.x<v.x);});
24     ll d;
25     mls<Point> hp;
26     d=euc(a[0], a[1]), hp.emplace(a[0]), hp.emplace(a[1]);
27     for(ll i=2, p=0; i<n; i++){
28         for(; p<n && (a[p].y-a[i].y)*(a[p].y-a[i].y)
    ↪ )>=d; hp.erase(hp.lwb(a[p])), p++);
29         auto it=hp.lwb(a[i]);
30         auto front=it, back=it;
31         forn(j, 0, 4){
32             if(front!=hp.ed())
    ↪ d=min(d, euc(a[i], *front));
33             else break; front++;
34         }
35         forn(j, 0, 4){
36             if(back!=hp.ed())
    ↪ d=min(d, euc(a[i], *back));
37             {if(back!=hp.bg()) back--; else
    ↪ break;}
38         }
39         hp.emplace(a[i]);
40     }
41     cout << d << '\n';
42 }

```

pick-theorem.cpp

```

1 void solve(istream &cin){
2     ll n; cin >> n;
3     vc<Point> a(n);
4     for(auto &v:a) cin >> v;
5     ll area=0, B=0;
6     forn(i, 0, n){
7         ll j=(i+1)%n;
8         area+=a[i]^a[j];
9         B+=gcd(abs(a[i].x-a[j].x), abs(a[i].y-a[j].y));
10    }
11    cout << (abs(area)/2+1-B/2) << ' ' << B << '\n';
12    //area=I+B/2-1 I=internal node B=boundary node
13 }

```

ray-casting.cpp

```

1 class Point{
2     public:
3         ll x, y;
4         Point(){}
5         Point(ll x, ll y):x(x), y(y){}
6         friend ll operator^(const Point a, const
    ↪ Point b){return a.x*b.y-a.y*b.x;}
7         friend ll operator*(const Point a, const
    ↪ Point b){return a.x*b.x+a.y*b.y;}
8         friend Point operator-(const Point a, const
    ↪ Point b){
9             return Point{a.x-b.x, a.y-b.y};
10        }
11        friend istream& operator>>(istream
    ↪ &in, Point &a){
12            in >> a.x >> a.y;
13            return in;

```

```

    }
};
bool on(Point &a, Point &b, Point &c){
    if((b-a)^(c-a)) return false;
    return 0<=((b-a)*(c-a)) &&
    ↪ ((b-a)*(c-a))<=((b-a)*(b-a));
}
void query(vc<Point> &a, Point &p){
    ll n=a.size(), ok=0;
    forn(i, 0, n){
        ll j=(i+1)%n;
        if(on(a[i], a[j], p)){cout << "BOUNDARY" <<
    ↪ '\n'; return;}
        if(p.y>a[i].y!=p.y>a[j].y){
            ll sg=(a[j]-a[i])^(p-a[i]);
            if(sg<0==a[i].y>a[j].y) ok^=1;
        }
    }
    cout << (ok&1?"INSIDE":"OUTSIDE") << '\n';
}

```

動態凸包.cpp

```

1 struct Point {
2     long long x, y;
3     bool operator<(const Point& p) const {
4         return x < p.x || (x == p.x && y < p.y);
5     }
6 };
7
8 long long cross(const Point& a, const Point& b, const
    ↪ Point& c) {
9     // 向量 ab x bc
10    return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x
    ↪ - a.x);
11 }
12
13 struct DynamicConvexHull {
14     set<Point> up, dn; // 上凸包、下凸包
15
16     void insert(const Point& p) {
17         insertHull(up, p, false); // 上凸包
18         insertHull(dn, p, true); // 下凸包
19     }
20
21     // 取得逆時針順序的凸包點 (不重複端點)
22     vector<Point> getConvexHull() const {
23         vector<Point> res;
24         for (auto p : up) res.push_back(p);
25         vector<Point> dnvec(dn.rbegin(), dn.rend());
26         for (int i = 1; i + 1 < dnvec.size(); ++i) //
    ↪ 去掉首尾重複
27             res.push_back(dnvec[i]);
28         return res;
29     }
30
31     private:
32         // flag: false= 上凸包, true= 下凸包
33         void insertHull(set<Point>& hull, const Point& p, bool
    ↪ flag) {
34             auto it = hull.insert(p).first;
35             // 刪除左側不在凸包上的點
36             while (it != hull.begin()) {
37                 auto prv = prev(it);
38                 if (prv == hull.begin()) break;
39                 auto pprv = prev(prv);
40                 if ((cross(*pprv, *prv, *it) > 0) ^ flag)
41                     hull.erase(prv);
42                 else break;
43             }
44             // 刪除右側不在凸包上的點
45             while (next(it) != hull.end()) {
46                 auto nxt = next(it);
47                 if (next(nxt) == hull.end()) break;
48                 auto nnxt = next(nxt);

```

```

49         if ((cross(*it, *nxt, *nnxt) > 0) ^ flag)
50             hull.erase(nxt);
51         else break;
52     }
53 }
54 };

```

多邊形重心計算.cpp

```

1 struct Point {
2     double x, y;
3 };
4
5 Point polygonCentroid(const vector<Point>& poly) {
6     double cx = 0, cy = 0, area = 0;
7     int n = poly.size();
8     for (int i = 0; i < n; ++i) {
9         double cross = poly[i].x * poly[(i + 1) % n].y -
10             poly[(i + 1) % n].x * poly[i].y;
11         cx += (poly[i].x + poly[(i + 1) % n].x) * cross;
12         cy += (poly[i].y + poly[(i + 1) % n].y) * cross;
13         area += cross;
14     }
15     area /= 2.0;
16     cx /= (6.0 * area);
17     cy /= (6.0 * area);
18     return {cx, cy};

```

找三點組成最大圓.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4 typedef pair<ll, ll> P;
5
6 ll cross(P a, P b) { return a.first * b.second - a.second * b.first; }
7 ll dist2(P a, P b) {
8     ll dx = a.first - b.first, dy = a.second - b.second;
9     return dx * dx + dy * dy;
10 }
11 double dist(P a, P b) { return sqrt(dist2(a, b)); }
12
13 // Andrew's Monotone Chain
14 vector<P> convex_hull(vector<P> pts) {
15     sort(pts.begin(), pts.end());
16     int n = pts.size(), k = 0;
17     vector<P> hull(2 * n);
18     for (int i = 0; i < n; ++i) {
19         while (k >= 2 &&
20             cross({hull[k-1].first-hull[k-2].first,
21                 hull[k-1].second-hull[k-2].second},
22                 {pts[i].first-hull[k-1].first,
23                 pts[i].second-hull[k-1].second}) <= 0)
24             --k;
25         hull[k++] = pts[i];
26     }
27     for (int i = n-2, t = k; i >= 0; --i) {
28         while (k > t &&
29             cross({hull[k-1].first-hull[k-2].first,
30                 hull[k-1].second-hull[k-2].second},
31                 {pts[i].first-hull[k-1].first,
32                 pts[i].second-hull[k-1].second}) <= 0)
33             --k;
34         hull[k++] = pts[i];
35     }
36     hull.resize(k);

```

```

31     return hull;
32 }
33
34 double circumradius(P a, P b, P c) {
35     double A = dist(b, c), B = dist(a, c), C = dist(a, b);
36     double S = abs(cross({b.first-a.first,
37         b.second-a.second},
38         {c.first-a.first,
39         c.second-a.second})) / 2.0;
40     if (S == 0) return 0; // 共線
41     return (A * B * C) / (4.0 * S);
42 }
43
44 int main() {
45     int n;
46     cin >> n;
47     vector<P> pts(n);
48     for (int i = 0; i < n; ++i) cin >> pts[i].first >>
49         pts[i].second;
50     vector<P> hull = convex_hull(pts);
51     int m = hull.size();
52     double ans = 0;
53     // 旋轉卡殼
54     for (int i = 0; i < m; ++i) {
55         int j = (i + 1) % m, k = (j + 1) % m;
56         do {
57             while (true) {
58                 int k2 = (k + 1) % m;
59                 double r1 = circumradius(hull[i], hull[j],
60                     hull[k]);
61                 double r2 = circumradius(hull[i], hull[j],
62                     hull[k2]);
63                 if (r2 > r1) k = k2;
64                 else break;
65             }
66             ans = max(ans, circumradius(hull[i], hull[j],
67                 hull[k]));
68             j = (j + 1) % m;
69         } while (j != i);
70     }
71     cout << fixed << setprecision(6) << ans << endl;

```

極角排序.cpp

```

1 struct Point {
2     int x, y;
3 };
4
5 // 以 base 為基準點，對 points 進行極角排序
6 void polarSort(vector<Point>& points, Point base) {
7     sort(points.begin(), points.end(), [&](const Point& a,
8         const Point& b) {
9         int dx1 = a.x - base.x, dy1 = a.y - base.y;
10        int dx2 = b.x - base.x, dy2 = b.y - base.y;
11        long long cross = 1LL * dx1 * dy2 - 1LL * dy1 *
12            dx2;
13        if (cross != 0) return cross > 0;
14        // 極角相同時，距離近的排前面
15        return dx1 * dx1 + dy1 * dy1 < dx2 * dx2 + dy2 *
16            dy2;
17    });

```

5 Data Structure

Array Version of Linked List.cpp

```

1 // 陣列版雙向串列。用前必須先 init 串好 (0 與 n+1 是 sentinel,
2 // 資料放 1..n),
3 // 否則 cut/insert 解參考 NULL 直接 segfault. cut(p): 把 p
4 // 從串列拔出並回傳;

```

```

3 // insert(p,q): 把 q 插到 p 右邊. sentinel 本身不可被 cut.
4 struct node{
5     struct node *nxt, *prv;
6     long long v;
7 }pos[100002];
8
9 void init(int n){ // 0 -> 1 -> ... -> n -> n+1
10     for(int i=0;i<=n+1;i++){
11         pos[i].nxt = (i<=n ? &pos[i+1] : NULL);
12         pos[i].prv = (i>=1 ? &pos[i-1] : NULL);
13     }
14 }
15
16 inline node* cut(node *p) // cut the node
17 {
18     p->nxt->prv = p->prv;
19     p->prv->nxt = p->nxt;
20     p->nxt = p->prv = NULL;
21     return p;
22 }
23
24 inline node* insert(node *p, node *q) // insert node q to
    ↪ right side of node p
25 {
26     q->nxt = p->nxt;
27     p->nxt->prv = q;
28     q->prv = p;
29     p->nxt = q;
30     return q;
31 }

```

Binary Index Tree.cpp

```

1 //BIT is always 1 index
2 ll n,t[mxN],a[mxN];
3 void built(){for(i,1,mxN){ t[i]=a[i]; if(i+(i&-i)<mxN)
    ↪ t[i+(i&-i)]+=t[i];}} //O(n)
4 void pad(ll p,ll val){for(;p<mxN;t[p]+=val,p+=p&-p);}
5 ll query(ll p){ll ans=0; for(;p;ans+=t[p],p=p&-p); return
    ↪ ans;}

```

Disjoint Set Union.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 class DisjointSets {
5     private:
6         vector<int> parents;
7         vector<int> sizes;
8
9     public:
10         DisjointSets(int size) : parents(size),
            ↪ sizes(size, 1) {
11             for (int i = 0; i < size; i++) {
12                 parents[i] = i; }
13
14         /** @return the "representative" node in x's
            ↪ component */
15         int find(int x) {
16             return parents[x] == x ? x : (parents[x] =
            ↪ find(parents[x]));
17
18         /** @return whether the merge changed connectivity
            ↪ */
19         bool unite(int x, int y) {
20             int x_root = find(x);
21             int y_root = find(y);
22             if (x_root == y_root) { return false; }
23
24             if (sizes[x_root] < sizes[y_root]) {
25                 swap(x_root, y_root); }

```

```

        sizes[x_root] += sizes[y_root];
        parents[y_root] = x_root;
        return true;
    }

    /** @return whether x and y are in the same
    ↪ connected component */
    bool connected(int x, int y) { return find(x) ==
    ↪ find(y); }
};

```

Segment Tree.cpp

```

1 //range add range query // sum query
2 #define forr(a,b,c) for(ll a=b;a<=c;--a)
3 #define forn(a,b,c) for(ll a=b;a<c;++a)
4 constexpr ll mxN=2e5+1;
5 #pragma GCC target("lzcnt")
6 ll n,q,t[mxN<<1],lz[mxN]{};
7 void built(){forr(i,n-1,1)t[i]=t[i<<1]+t[i<<1|1];}
8 void apply(ll p,ll k,ll val){t[p]+=val*k; if(p<n)
    ↪ lz[p]+=val;}
9 void push(ll p){forr(h=63-__builtin_clzll(p);h;h--)
    ↪ if(lz[p>>h]){apply((p>>h)<<1,1<<(h-1),lz[p>>h]),apply(
    ↪ (p>>h)<<1|1,1<<(h-1),lz[p>>h]),lz[p>>h]=0;}}
10 void built(ll p){for(;p>=1;){if(!lz[p])
    ↪ t[p]=t[p<<1]+t[p<<1|1];}}
11 void rad(ll l,ll r,ll val){
12     ll l0=l+n-1,r0=r+n-1,k=1; push(l0),push(r0);
13     for(;l<=r;l>=1,r>=1,k<=1){if(l&1) apply(l++,k,val);
    ↪ if(r&1^1) apply(r--,k,val);}
14     built(l0),built(r0);
15 }
16 ll query(ll l,ll r){ ll ans=0; push(l+n-1),push(r+n-1);
    ↪ for(;l<=r;l>=1,r>=1){if(l&1) ans+=t[l++]; if(r&1^1)
    ↪ ans+=t[r--];} return ans;} //range query
17 ll query(ll p){push(p+n-1); return t[p];} // point query

```

persistence-seg-tree.cpp

```

1 // 節點數: build用2n-1, 每次單點改約log(n)+1個; mxT要 >=
    ↪ 2n+q*20 (n=q=2e5時約4.2e6)
2 // 記憶體約 7e6*(8+4+4) 106MB; 多測時 t/lc/rc 殘留值會錯,
    ↪ 需清到 sz 為止
3 constexpr ll mxN=2e5+1;
4 constexpr ll mxT=7e6+1;
5 ll n,q,t[mxT],a[mxN],rt[mxN<<1],rsc,sz; int
    ↪ lc[mxT],rc[mxT];
6 void built(ll i,ll sl,ll sr){
7     if(sl==sr){t[i]=a[sl]; return;}
8     ll mid; mid=(sl+sr)>>1;
9     lc[i]=++sz,rc[i]=++sz;
10    built(lc[i],sl,mid);
11    built(rc[i],mid+1,sr);
12    t[i]=t[lc[i]]+t[rc[i]];
13 }
14 ll pst(ll pos,ll val,ll i,ll sl,ll sr){
15     ll x; x=++sz;
16     if(sl==sr){t[x]=val; return x;}
17     ll mid; mid=(sl+sr)>>1;
18     if(pos<=mid)
19         ↪ lc[x]=pst(pos,val,lc[i],sl,mid),rc[x]=rc[i];
20     else lc[x]=lc[i],rc[x]=pst(pos,val,rc[i],mid+1,sr);
21     t[x]=t[lc[x]]+t[rc[x]];
22     return x;
23 }
24 ll query(ll l,ll r,ll i,ll sl,ll sr){
25     if(l>sr || r<sl) return 0;
26     if(l<=sl && sr<=r) return t[i];
27     ll mid; mid=(sl+sr)>>1;
28     return query(l,r,lc[i],sl,mid)+query(l,r,rc[i],mid
    ↪ +1,sr);
}

```

```

29 void solve(istream &cin){
30     cin>>n>>q;
31     sz=rsc=0;
32     forn(i,1,n+1) cin>>a[i];
33     rt[++rsc]=++sz;
34     built(rt[1],1,n);
35     while(q--){
36         ll op; cin>>op;
37         if(op==3){
38             ll k; cin>>k;
39             rt[++rsc]=++sz;
40             lc[rt[rsc]]=lc[rt[k]],rc[rt[rsc]]=rc[rt[k]]
                ↪ rc[rt[k]],t[rt[rsc]]=t[rt[k]]
41         }
42         else if(op==1){
43             ll k,a,x; cin>>k>>a>>x;
44             rt[k]=pst(a,x,rt[k],1,n);
45         }
46         else{
47             ll k,a,b; cin>>k>>a>>b;
48             cout << query(a,b,rt[k],1,n) <<
                ↪ '\n';
49         }
50     }
51 }
52

```

splay-tree.cpp

```

1 constexpr ll mxN=2e5+3;
2 ll n,m,rt,t[mxN][2],cnt[mxN],val[mxN],f[mxN],lz[mxN],v[]
    ↪ als[mxN];
3 string s;
4 ll dir(ll x){return t[f[x]][1]==x;}
5 void push_down(ll x){
6     if(lz[x]){
7         swap(t[x][0],t[x][1]);
8         if(t[x][0]) lz[t[x][0]]^=1;
9         if(t[x][1]) lz[t[x][1]]^=1;
10        lz[x]=0;
11    }
12 }
13 void push(ll x){cnt[x]=cnt[t[x][0]]+1+cnt[t[x][1]],vals[x]
    ↪ =vals[t[x][0]]+val[x]+vals[t[x][1]];}
14 void rotate(ll x){
15     ll y=f[x],z=f[y],d=dir(x);
16     t[y][d]=t[x][d^1],t[x][d^1]=y;
17     if(t[y][d]) f[t[y][d]]=y;
18     if(z) t[z][dir(y)]=x;
19     f[y]=x,f[x]=z,push(y),push(x);
20 }
21 void splay(ll &z,ll x){
22     ll w=f[z];
23     for(ll y;(y=f[x])!=w;rotate(x)){
24         if(f[y]!=w) rotate(dir(x)==dir(y)?y:x);
25     }
26     z=x;
27 }
28 void build(){
29     rt=0;
30     forn(i,1,n+3){
31         t[i][0]=rt;
32         if(rt) f[rt]=i;
33         rt=i;
34     }
35     splay(rt,1);
36 }
37 void loc(ll &z,ll k){
38     ll x=z;
39     for(push_down(x);cnt[t[x][0]]+1!=k;push_down(x)){
40         if(k>cnt[t[x][0]])
            ↪ k-=cnt[t[x][0]]+1,x=t[x][1];
41         else x=t[x][0];
42     }
43     splay(z,x);
44 }

```

```

45 void opt(ll l,ll r){
46     loc(rt,l);
47     loc(t[rt][1],r-l+2);
48     ll x=t[t[rt][1]][0];
49     lz[x]^=1;
50 }
51 ll query(ll l,ll r){
52     loc(rt,l),loc(t[rt][1],r-l+2);
53     ll x=t[t[rt][1]][0],y=vals[x];
54     return y;
55 }
56 void solve(istream &cin){
57     cin >> n >> m;
58     forn(i,2,n+2) cin >> val[i];
59     build();
60     forn(i,0,m){
61         ll op,l,r; cin >> op >> l >> r;
62         if(op==1) opt(l,r);
63         else cout << query(l,r) << '\n';
64     }
65 }

```

6 Not that important

EPS.cpp

```

1 #include <cmath>
2 const double EPS = 1e-9;
3
4 bool eq(double a, double b) {
5     return fabs(a - b) < EPS;
6 }
7
8 bool lt(double a, double b) {
9     return a < b - EPS;
10 }
11
12 bool le(double a, double b) {
13     return a < b + EPS;
14 }

```

Gauce Elimination.cpp

```

1 // a[N][N] 是增广矩阵
2 int gauss()
3 {
4     int c, r;
5     for (c = 0, r = 0; c < n; c++)
6     {
7         int t = r;
8         for (int i = r; i < n; i++) //
            ↪ 找到绝对值最大的行
9             if (fabs(a[i][c]) > fabs(a[t][c]))
                t = i;
10
11         if (fabs(a[t][c]) < eps) continue;
12
13         for (int i = c; i <= n; i++) swap(a[t][i],
            ↪ a[r][i]); // 将绝对值最大的行换到最顶端
14         for (int i = n; i >= c; i--) a[r][i] /= a[r][c];
            ↪ // 将当前行的首位变成1
15         for (int i = r + 1; i < n; i++) //
            ↪ 用当前行将下面所有的列消成0
16             if (fabs(a[i][c]) > eps)
                for (int j = n; j >= c; j--)
                    a[i][j] -= a[r][j] * a[i][c];
17
18         r++;
19     }
20
21     if (r < n)
22     {
23
24
25

```



```

26     for (int i = r; i < n; i ++ )
27         if (fabs(a[i][n]) > eps)
28             return 2; // 无解
29     return 1; // 有无穷多组解
30 }
31
32 for (int i = n - 1; i >= 0; i -- )
33     for (int j = i + 1; j < n; j ++ )
34         a[i][n] -= a[i][j] * a[j][n];
35
36 return 0; // 有唯一解
37 }

```

Joseph.cpp

```

1 // 回傳 0-indexed 的倖存者位置
2 ll josephus(ll n, ll k) { // O(n)
3     ll res = 0;
4     for (ll i = 1; i <= n; ++i) res = (res + k) % i;
5     return res;
6 }
7 ll josephus2(ll n, ll k) { // O(k log n), k<=n 時用; k>n
8     ↪ 落回迴圈版避免深遞迴爆棧
9     if (n == 1) return 0;
10    if (k == 1) return n - 1;
11    if (k > n) {
12        ll res = 0;
13        for (ll i = 2; i <= n; ++i) res = (res + k) % i;
14        return res;
15    }
16    ll res = josephus2(n - n / k, k);
17    res -= n % k;
18    if (res < 0)
19        res += n; // mod n
20    else
21        res += res / (k - 1); // 还原位置
22    return res;
23 }

```

7 New

Convex_hull_trick.cpp

```

1 // 特定題解法: 斜率=(y2-y1)/m 是整數除法(負數向零截斷),
2 ↪ 查詢x需單調遞增
3 // check() 乘法沒用__int128, c差*m差 ~1e18 以上會溢位,
4 ↪ 改編他題時注意
5 class line{
6     public:
7         ll m,c;
8         line(){ }
9         line(ll m,ll c):m(m),c(c){ }
10        friend bool operator<(const line &a,const
11        ↪ line &b){
12            return a.m<b.m || (a.m==b.m &&
13            ↪ a.c>b.c);
14        }
15        ll cal(ll x){return m*x+c;}
16    };
17    constexpr ll mxN=1e5+1;
18    ll n,m,id=0,st[mxN];
19    vc<line> ln;
20    bool check(ll i,ll j,ll k){
21        return (ln[i].c-ln[k].c)*(ln[j].m-ln[i].m)<(ln[i].
22        ↪ c-ln[j].c)*(ln[k].m-ln[i].m);
23    }
24    void solve(istream &cin){
25        cin >> n >> m;
26        forn(i,0,n){
27            ll y1,y2; cin >> y1 >> y2;
28            ln.emp(line{(y2-y1)/m,y1});
29        }

```

```

25    sort(all(ln)),ln.resize(unique(all(ln),[])(auto
26    ↪ a,auto b){return a.m==b.m;})-ln.bg());
27    forn(i,0,ln.size()){
28        while(id>1 && check(st[id-1],st[id],i))
29            ↪ id--;
30        st[++id]=i;
31    }
32    ll ptr=1;
33    forn(i,0,m+1){
34        while(ptr+1<=id && ln[st[ptr+1]].cal(i)>ln[
35        ↪ st[ptr]].cal(i))
36            ↪ ptr++;
37        cout << ln[st[ptr]].cal(i) << " \n"[i==m];
38    }
39 }

```

Ray_casting.cpp

```

1 class Point{
2     public:
3         ll x,y;
4         Point(){ }
5         Point(ll x,ll y):x(x),y(y){ }
6         friend Point operator-(const Point
7         ↪ &a,const Point &b){return
8         ↪ {a.x-b.x,a.y-b.y};}
9         friend ll operator*(const Point &a,const
10        ↪ Point &b){return a.x*b.x+a.y*b.y;}
11        friend ll operator^(const Point &a,const
12        ↪ Point &b){return a.x*b.y-a.y*b.x;}
13    };
14    ll n,m;
15    vc<Point> p;
16    bool on(Point &a,Point &b,Point &c){
17        if((b-a)^(c-a)) return false;
18        return 0<=((b-a)*(c-a)) &&
19        ↪ ((b-a)*(c-a))<=((b-a)*(b-a));
20    }
21    void query(Point &x){
22        ll ok=0;
23        forn(i,0,n){
24            if(on(p[i],p[(i+1)%n],x)){ cout <<
25            ↪ "BOUNDARY" << '\n';return;}
26            Point &a=p[i],&b=p[(i+1)%n];
27            if(a.y>x.y != b.y>x.y){
28                ll sg=(x-a)^(b-a);
29                if(b.y>a.y==(sg<0)) ok^=1;
30            }
31        }
32        cout << (ok&1?"INSIDE":"OUTSIDE") << '\n';
33    }

```

block_cut_tree.cpp

```

1 constexpr ll mxN=1e5+1;
2 constexpr ll mxB=20;
3 ll n,m,q,st[mxN],id=0,mark[mxN]{},dfn[mxN]{},low[mxN],tim=
4 ↪ 0,nid[mxN],pa[mxN<<1][mxB],d[mxN<<1];
5 vc<ll> adj[mxN];
6 vc<vc<ll>> cmp,t;
7 void tarjan(ll u=1,ll p=0){
8     dfn[u]=low[u]=++tim,st[++id]=u;
9     for(auto&v:adj[u]){
10        if(v==p) continue;
11        if(dfn[v]) low[u]=min(low[u],dfn[v]);
12        else{
13            tarjan(v,u);
14            low[u]=min(low[u],low[v]);
15            if(low[v]>=dfn[u]){
16                mark[u]=(dfn[u]>1 ||
17                ↪ dfn[v]>2);
18                cmp.emp(vc<ll>{u});

```

```

17         while(cmp.back().back()!=v)
18             ↪ )
19         ↪ cmp.back().emp(st[id--]);
20     }
21 }
22 void dfs(ll u=1,ll p=0,ll lv=1){
23     d[u]=lv,pa[u][0]=p;
24     for(auto&v:t[u]){
25         if(v!=p) dfs(v,u,lv+1);
26     }
27 }
28 void built(){
29     ll x=0;
30     t.emp(vc<ll>{});
31     forn(i,1,n+1) if(mark[i])
32         ↪ nid[i]++,t.emp(vc<ll>{});
33     for(auto&a:cmp){
34         ll y=++x;
35         t.emp(vc<ll>{});
36         for(auto&u:a){
37             if(mark[u]) t[nid[u]].emp(y),t[y].emp(nid[u]);
38             ↪ emp(nid[u]);
39             else nid[u]=y;
40         }
41     }
42     dfs();
43     forn(j,1,mxB) forn(i,1,t.size())
44         ↪ pa[i][j]=pa[pa[i][j-1]][j-1];
45 }
46 ll lca(ll a,ll b){
47     if(d[a]<d[b]) swap(a,b);
48     forr(i,mxB-1,0) if(d[a]-(1<<i)>=d[b]) a=pa[a][i];
49     if(a==b) return a;
50     forr(i,mxB-1,0) if(pa[a][i]!=pa[b][i])
51         ↪ a=pa[a][i],b=pa[b][i];
52     return pa[a][0];
53 }
54 ll query(ll a,ll b,ll c){
55     ll lca1=lca(a,b),lca2=lca(a,c),lca3=lca(b,c);
56     return (lca1==c || (lca2==c && lca3==lca1) ||
57             ↪ (lca3==c && lca2==lca1));
58 }
59 void solve(){
60     cin >> n >> m >> q;
61     forn(i,0,m){
62         ll u,v; cin >> u >> v;
63         adj[u].emp(v),adj[v].emp(u);
64     }
65     tarjan();
66     built();
67     while(q--){
68         ll a,b,c; cin >> a >> b >> c;
69         cout << ((mark[c] &&
70             ↪ query(nid[a],nid[b],nid[c])) || a==c
71             ↪ || b==c?"NO":"YES") << '\n';
72     }
73 }

```

cross_dot_product.cpp

```

1 //A=(x1,y1),B=(x2,y2)
2 //A cross B = ABsin x = x1*y2-y1*x2
3 //A dot B =AB cos x = x1*x2+y1*y2

```

dec_string_to_bitset.cpp

```

1 // 10進位string(只含0-9, 長度<=500) 轉 bitset 二進位,
2 ↪ 支援先減一個小常數
3 // decSub(s,x): 回傳 s-x (x>=0, 需保證 s>=x), O(len)
4 // decToBits(s): 10進string -> bitset, b[0]為LSB, O(len~2)
5 // bitsToDec(b): bitset -> 10 進 string, 驗證/輸出用

```

```

// 500 位 10 進 1661 bits, mxB=2048 夠用
constexpr ll mxB=2048;
string decSub(string s,ll x){
    for(ll i=s.size()-1;i>=0&&x;i--){
        ll d=(s[i]-'0')-x%10; x/=10;
        if(d<0){d+=10; x++;}
        s[i]=d+'0';
    }
    ll p=0; while(p+1<(ll)s.size()&&s[p]=='0') p++;
    return s.substr(p);
}
bitset<mxB> decToBits(string s){
    bitset<mxB> b;
    for(ll i=0;s!="0";i++){
        b[i]=(s.back()=='0')&1;
        ll r=0;
        for(char &c:s){ll d=r*10+(c-'0');
            ↪ c='0'+d/2; r=d&1;}
        ll p=0; while(p+1<(ll)s.size()&&s[p]=='0')
            ↪ p++;
        s=s.substr(p);
    }
    return b;
}
string bitsToDec(bitset<mxB> b){
    string s="0";
    ll hi=mxB-1; while(hi>0&&!b[hi]) hi--;
    forr(i,hi,0){
        ll c=b[i];
        forr(j,(ll)s.size()-1,0){ll
            ↪ d=(s[j]-'0')*2+c; s[j]='0'+d%10;
            ↪ c=d/10;}
        if(c) s=char('0'+c)+s;
    }
    return s;
}
void solve(istream &cin){
    string s; ll k;
    cin>>s>>k; // 10 進 string 與 要減的常數
    bitset<mxB> b=decToBits(decSub(s,k));
    ll hi=mxB-1; while(hi>0&&!b[hi]) hi--;
    forr(i,hi,0) cout<<b[i]; // MSB->LSB 印出 2 進位
    cout<<'\n';
}

```

dominator_tree.cpp

```

1 constexpr ll mxN=1e5+1;
2 ll n,m,tim=0,id[mxN],sd[mxN],pa[mxN],f[mxN],lb[mxN],et[mxN];
3 ↪ ret[mxN];
4 vc<ll> adj[mxN],radj[mxN],t[mxN],st;
5 void dfs(ll u=1){
6     et[u]=++tim,ret[tim]=u;
7     id[tim]=sd[tim]=lb[tim]=pa[tim]=tim;
8     for(auto&v:adj[u]){
9         if(!et[v]) dfs(v),f[et[v]]=et[u];
10        radj[et[v]].emp(et[u]);
11    }
12 }
13 ll find(ll u,ll x=0){
14     if(u==pa[u]) return x?-1:u;
15     ll v=find(pa[u],x+1);
16     if(v<0) return u;
17     if(sd[lb[u]]>sd[lb[pa[u]]]) lb[u]=lb[pa[u]];
18     pa[u]=v;
19     return x?v:lb[u];
20 }
21 void built(){
22     vc<ll> arr[n+1];
23     forr(i,n,1){
24         for(auto&v:radj[i])
25             ↪ sd[i]=min(sd[i],sd[find(v)]);
26         if(i>1) arr[sd[i]].emp(i);
27         for(auto&v:arr[i]){
28             ll x=find(v);
29             if(sd[x]==sd[v]) id[v]=sd[v];
30         }
31     }
32 }

```



```

28         else id[v]=x;
29     }
30     if(i>1) pa[i]=f[i];
31 }
32 forn(i,2,n+1){
33     if(id[i]!=sd[i]) id[i]=id[id[i]];
34     t[ret[i]].emp(ret[id[i]]),t[ret[id[i]]].emp
        ↪ p(ret[i]);
35 }
36 }
37 void dfs_ans(ll u=1,ll p=0){
38     st.emp(u);
39     if(u==n){
40         sort(all(st));
41         cout << st.size() << '\n';
42         for(auto&v:st) cout << v << ' '; cout <<
        ↪ '\n';
43         exit(0);
44     }
45     for(auto&v:t[u]){
46         if(v==p) continue;
47         dfs_ans(v,u);
48     }
49     st.pop_back();
50 }
51 void solve(istream &cin){
52     cin>>n>>m;
53     forn(i,1,m+1){
54         ll u,v; cin>>u>>v;
55         adj[u].emp(v);
56     }
57     dfs();
58     built();
59     dfs_ans();
60 }

```

extra.cpp

```

1 //__builtin_popcountll
2 //__builtin_parityll
3 //__builtin_clzll
4 //__builtin_ctzll
5 // mt19937 rng(chrono::steady_clock::now().time_since_epoch
    ↪ h().count());
6 // const ll M = 991831889;
7 // const ll C = uniform_int_distribution<ll>(0.1 * M, 0.9
    ↪ M)(rng);
8

```

fft.cpp

```

1 class FFT{
2     public:
3         ll mod,root,root_inv,maX,M,M_inv;
4         FFT(){
5             FFT(ll a,ll b,ll c,ll d,ll e,ll
        ↪ f):mod(a),root(b),root_inv(c),maX(d),M
        ↪ (e),M_inv(f){}
6         ll fsp(ll a,ll m){
7             ll ans=1;
8             while(m){
9                 if(m&1) ans=(ans*a)%mod;
10                a=(a*a)%mod,m>>=1;
11            }
12            return ans;
13        }
14        void fft(ll sz,vc<ll> &arr,ll inv){
15            for(ll i=1,j=0;i<sz;++i){
16                ll bit=sz>>1;
        ↪ for(;j&bit;bit>>=1)
        ↪ j^=bit; j^=bit;
17                if(i<j)
        ↪ swap(arr[i],arr[j]);
18            }

```

```

        for(ll l=2;l<=sz;l<=1){
            ll wlen=inv?root_inv:root;
            for(ll i=1;i<maX;i<=1)
        ↪ wlen=(wlen*wlen)%mod;
            for(ll i=0;i<sz;i+=l){
                ll w=1;
                for(ll
        ↪ j=0;j<l/2;++j){
                    ll x=arr[i
        ↪ +j],y=
        ↪ (w*arr
        ↪ [i+j+l
        ↪ /2])%m
        ↪ od;
                    arr[i+j]=
        ↪ x+y<mo
        ↪ d?x+y:
        ↪ x+y-mo
        ↪ d);
                    arr[i+j+l
        ↪ 2]=(x-
        ↪ y>0?x
        ↪ -y:x-y
        ↪ +mod);
                    w=(w*wlen)
        ↪ %mod;
                }
            }
        }
        if(inv){
            ll n_1=fsp(sz,mod-2);
            forn(i,0,sz){
                arr[i]=(arr[i]*n_1
        ↪ )%mod;
            }
        }
    }
}

```

```

};
constexpr ll MM=1002772198720536577;
//
//g=3
//998244353=119*2^23 + 1
//g^-1=15311432
//inv=469870224
//M1=1004535809
//M1^-1=332747959
//
//1004535809=479*2^21 + 1
//g^-1=702606812
//inv=700146880
//M2=998244353
//M2^-1=669690699
ll mul(ll a,ll b){
    __int128 aa=a,bb=b,cc;
    cc=(aa*bb)%MM;
    return (ll)cc;
}
void solve(){
    FFT fft1(998244353,15311432,469870224,1<<23,100453
    ↪ 5809,332747959);
    FFT fft2(1004535809,702606812,700146880,1<<21,9982
    ↪ 44353,669690699);
    //ll x=(fft1.M*fft1.M_inv)%fft1.mod;
    //ll y=(fft2.M*fft2.M_inv)%fft2.mod;
    //err(x,y);
    ll n,m;
    cin >> n >> m;
    vc<ll> a(n+1),b(m+1);
    for(auto &v:a) cin >> v;
    for(auto &v:b) cin >> v;
    ll sz=1;
    for(;sz<a.size()+b.size();sz<=1);
    a.resize(sz),b.resize(sz);
    vc<ll> A=a,B=b;
    fft1.fft(sz,a,0);
    fft1.fft(sz,b,0);
    forn(i,0,sz) a[i]=(a[i]*b[i])%fft1.mod;
    fft1.fft(sz,a,1);

```

```

78     fft2.fft(sz,A,0);
79     fft2.fft(sz,B,0);
80     forn(i,0,sz) A[i]=(A[i]*B[i])%fft2.mod;
81     fft2.fft(sz,A,1);
82
83     forn(i,0,n+m+1) cout <<
84     ↪ (mul(a[i],fft1.M*fft1.M_inv) +
85     ↪ mul(A[i],fft2.M*fft2.M_inv))%MM << "
86     ↪ "\n"[i==n+m];
87 }

```

fwf.cpp

```

1  constexpr ll mxB=21;
2  ll a[1<<mxB]{};
3  void fwf(ll *arr,ll type){
4      for(ll k=2;k<=(1<<mxB);k<=1){ //必須<=,
5          ↪ 否則最高層蝶形沒做, 值>=2^(mxB-1)會錯
6          for(ll i=0,m=k>>1;i+m<(1<<mxB);i+=k){
7              forn(j,0,m){
8                  ll x,y; x=arr[i+j],y=arr[i+j+m];
9                  ↪ +j+m];
10                 arr[i+j]=x+y,arr[i+j+m]=x-y;
11                 ↪ y;
12                 if(type) arr[i+j]/=2,arr[i+j+m]/=2;
13             }
14         }
15     }
16 }
17 void solve(istream &cin){
18     ll n,pfx,mx;
19     cin>>n;
20     mx=pfx=0;
21     a[0]=1;
22     forn(i,0,n){
23         ll x; cin>>x;
24         pfx^=x,a[pfx]++,mx=max(mx,a[pfx]);
25     }
26     fwf(a,0);
27     forn(i,0,1<<mxB) a[i]=a[i]*a[i];
28     fwf(a,1);
29     ll res; res=0;
30     // forn(i,0,17) cout << a[i] << "\n"[i==16];
31     forn(i,0,1<<mxB) res+=(!i?mx>1:a[i]>0);
32     cout << res << "\n";
33     forn(i,0,1<<mxB) if(!i?mx>1:a[i]) cout << i << " ";
34     cout << "\n";
35 }

```

kmp-automaton.cpp

```

1  void kmp(){
2      forn(i,1,m){
3          ll j; j=k[i-1];
4          while(j>0 && s[j]!=s[i]) j=k[j-1];
5          if(s[j]==s[i]) k[i]=j+1;
6      }
7  }
8  void solve(istream &cin){
9      cin>>n>>s;
10     m=s.length();
11     kmp();
12     //forn(i,0,m){err(i,k[i]);}
13     forn(i,0,m+1){
14         forn(j,0,26){
15             if(i>0 && (s[i]-'A')!=j)
16                 ↪ nex[i][j]=nex[k[i-1]][j];
17             else nex[i][j]=i+(s[i]-'A'==j);
18         }
19     }
20     // forn(i,0,m+1){forn(j,0,26){err(i,j,nex[i][j]);}}

```

```

21     dp[0][0][0]=1;
22     forn(i,0,n){
23         forn(f,0,2){
24             forn(j,0,m+1){
25                 forn(c,0,26){
26                     // err(i,j,c,nex[j][c]);
27                     ↪ ][c],dp[i][j]);
28                     ll nf; nf=f|(nex[j][c]==m);
29                     dp[i+1][nex[j][c]]
30                     ↪ [nf]=(dp[i+1][nf]);
31                     ↪ nex[j][c][nf];
32                     ↪ +dp[i][j][f])%
33                     ↪ md11;
34                 }
35             }
36         }
37     }
38     ll res; res=0;
39     forn(i,0,m+1) res=(res+dp[n][i][1])%md11;
40     cout << res << "\n";
41 }

```

li_chao_tree.cpp

```

1  class line {
2  public:
3      ll m,c;
4      line(){m=0,c=-inf-1;}
5      line(ll m,ll c):m(m),c(c){}
6      ll cal(ll x){return m*x+c;}
7  };
8  constexpr ll mxN=1e5+5;
9  ll n;
10 line t[mxN<<2];
11 void insert(line x,ll l,ll r,ll i,ll sl,ll sr){
12     if(sr<l || r<sl || l>r) return;
13     ll mid=(sl+sr)>>1;
14     if(l<=sl && sr<=r){ //if extended otherwise remove
15         ↪ this condition
16         bool lf=t[i].cal(sl-1)<x.cal(sl-1),mf=t[i].cal(sl-1)<x.cal(mid-1),rf=t[i].cal(sr-1)<x.cal(sr-1);
17         if(mf) swap(x,t[i]);
18         if(sl==sr) return;
19         if(lf^mf) insert(x,l,r,i<<1,sl,mid);
20         if(rf^mf) insert(x,l,r,i<<1|1,mid+1,sr);
21         return;
22     }
23     insert(x,l,r,i<<1,sl,mid);
24     insert(x,l,r,i<<1|1,mid+1,sr);
25 }
26 ll query(ll x,ll i,ll sl,ll sr){
27     ll mid=(sl+sr)>>1;
28     if(sl==sr) return t[i].cal(x-1);
29     if(x<=mid) return
30     ↪ max(t[i].cal(x-1),query(x,i<<1,sl,mid));
31     else return
32     ↪ max(t[i].cal(x-1),query(x,i<<1|1,mid+1,sr));
33 }
34 void solve(){
35     cin >> n;
36     forn(i,0,n){
37         ll op;
38         cin >> op;
39         if(op==1){
40             ll a,b,l,r;
41             cin >> a >> b >> l >> r;
42             line x{a,b};
43             insert(x,l+1,r+1,1,1,mxN-4);
44         }
45         else{
46             ll x;
47             cin >> x;

```

```

45         x=query(x+1,1,1,mxN-4);
46         if(x>-inf-1) cout << x << '\n';
47         else cout << "NO" << '\n';
48     }
49 }
50 }

```

manacher.cpp

```

1  ll dp[mxN][2]{},n,res[mxN];
2  void solve(istream &cin){
3      cin >> s,n=s.length();
4      ll l1,l2,r1=-1,r2=-1;
5      forn(i,0,n) res[i]=1;
6      forn(i,0,n){
7          if(r1>i) dp[i][1]=min(r1-i,dp[l1+r1-i][1]);
8          if(r2>i)
9              ⇨ dp[i][0]=min(r2-i,dp[l2+r2-i-1][0]);
10         while(i+dp[i][1]<n && i-dp[i][1]>=0 &&
11             ⇨ s[i+dp[i][1]]==s[i-dp[i][1]])
12             ⇨ res[i+dp[i][1]]=max(res[i+dp[i][1]],2*
13             ⇨ dp[i][1]+1),dp[i][1]++;
14         while(i+dp[i][0]+1<n && i-dp[i][0]>=0 &&
15             ⇨ s[i+dp[i][0]+1]==s[i-dp[i][0]])
16             ⇨ res[i+dp[i][0]+1]=max(res[i+dp[i][0]+1],
17             ⇨ 2*(dp[i][0]+1)),dp[i][0]++;
18         if(i+dp[i][1]>r1)
19             ⇨ l1=i-dp[i][1],r1=i+dp[i][1];
20         if(i+dp[i][0]>r2)
21             ⇨ l2=i-dp[i][0]+1,r2=i+dp[i][0];
22     }
23     forn(i,0,n) cout << res[i] << " \n"[i==n-1];
24 }

```

shoelace.cpp

```

1  //consecutive point p[i] (0<=i<n)
2  //注意: 輸出是「2倍面積」(鞋帶公式沒除2); 2A =
3  ⇨ sum(p[i]^p[i+1])
4  //| 座標 |<=1e9 時 2A<=8e18 勉強不溢位, 更大要 __int128
5
6  void solve(istream &cin){
7      ll n,res=0;
8      cin>>n;
9      forn(i,1,n+1){
10         cin>>a[i];
11         if(i>1) res+=a[i-1]^a[i]; //cross product
12         if(i==n) res+=a[n]^a[1];
13     }
14     cout << abs(res) << '\n';
15 }

```

suffix_automaton.cpp

```

1  constexpr ll mxN=1e5+1;
2  class state{
3  public:
4      ll len,link,next[26];
5
6      state(){
7      };
8      ll n,sz;
9      state t[mxN<<1];
10     void sam(const string &s){
11         memset(t,0,sizeof(t));
12         t[0].link=-1;
13         ll last,cur; last=0;
14         for(auto&v:s){
15             cur=++sz;
16             t[cur].len=t[last].len+1;
17             ll p; p=last;

```

```

while(p!=-1 && !t[p].next[v-'a'])
    ⇨ t[p].next[v-'a']=cur,p=t[p].link;
if(p==-1) t[cur].link=0;
else{
    ll q; q=t[p].next[v-'a'];
    if(t[p].len+1==t[q].len) t[cur].link=q;
    else{
        ll clone; clone=++sz;
        t[clone].len=t[p].len+1,t[clone].link=t[q].
        ⇨ .link;
        memcpy(t[clone].next,t[q].next,sizeof(t[q].
        ⇨ .next));
        while(p!=-1 && t[p].next[v-'a']==q)
            ⇨ t[p].next[v-'a']=clone,p=t[p].link;
        t[q].link=t[cur].link=clone;
    }
}
last=cur;
}
}
void solve(istream &cin){
    string s;
    cin>>s;
    n=s.length(),sz=0;
    sam(s);
    vc<ll> res(n+2,0);
    forn(i,1,sz+1)
        ⇨ res[t[t[i].link].len+1]++,res[t[i].len+1]--;
    partial_sum(all(res),res.bg());
    forn(i,1,n+1) cout << res[i] << " \n"[i==n];
}

```

suffixarray_and_lcp.cpp

```

1  constexpr ll mxN=1e5+5;
2  ll sa[mxN],nsa[mxN],c[mxN],nc[mxN],cnt[mxN] {},lcp[mxN];
3  void sfa(string s){
4      s+="$";
5      ll n=s.length();
6      memset(cnt,0,sizeof(cnt)); //cnt有殘留,
7      ⇨ 不清第二次呼叫必錯
8      forn(i,0,n) cnt[s[i]]++; forn(i,1,256)
9      ⇨ cnt[i]+=cnt[i-1];
10     forr(i,n-1,0) sa[--cnt[s[i]]]=i; c[sa[0]]=0;
11     ⇨ forn(i,1,n)
12     ⇨ c[sa[i]]=c[sa[i-1]]+(s[sa[i]]!=s[sa[i-1]]);
13     for(ll h=0;(1<=h)<n;h++){
14         forn(i,0,n) nsa[i]=((sa[i]-(1<=h))%n+n)%n;
15         fill(cnt,cnt+c[sa[n-1]]+1,0);
16         forn(i,0,n) cnt[c[nsa[i]]]++;
17         ⇨ forn(i,1,c[sa[n-1]]+1)
18         ⇨ cnt[i]+=cnt[i-1];
19         forr(i,n-1,0) sa[--cnt[c[nsa[i]]]]=nsa[i];
20         nc[sa[0]]=0;
21         forn(i,1,n){
22             pll pre={c[sa[i-1]],c[(sa[i-1]+(1<
23             ⇨ <h))%n]},cur={c[sa[i]],c[(sa[i]
24             ⇨ ]+(1<=h))%n]};
25             nc[sa[i]]=nc[sa[i-1]]+(cur!=pre);
26         }
27         swap(c,nc);
28     }
29 }
30 void Lcp(const string &s){
31     ll n=s.length(),k=0;
32     forn(i,0,n){
33         if(c[i]>n-1){k=0; continue;}
34         ll j=sa[c[i]+1];
35         for(;i+k<n && j+k<n && s[i+k]==s[j+k];k++);
36         lcp[c[i]]=k;
37         if(k) k--;
38     }
39 }

```

xor_basis.cpp

```

1 // 值需 < 2^31; 值域到 1e18 時 mxB 改 60、迴圈上限改 59
2 constexpr ll mxN=2e5+1;
3 constexpr ll mxB=31;
4 ll n,a[mxN],b[mxB]{};
5 void built(){
6     forn(i,1,n+1){
7         ll x; x=a[i];
8         forr(j,30,0){
9             if((x>>j)&1){
10                 if(!b[j]){b[j]=x; break;}
11                 else x^=b[j];
12             }
13         }
14     }
15 }
16 void solve(istream &cin){
17     cin>>n;
18     forn(i,1,n+1) cin>>a[i];
19     built();
20     ll res; res=0;
21     forr(i,30,0){
22         if(res>>i&1^1) res^=b[i];
23     }
24     cout << res << '\n';
25 }

```

zfunction_and_kmp.cpp

```

1 constexpr ll mxN=1e6+1;
2 vc<ll> zf(const string &s){
3     ll n,l,r; n=s.length(),l=r=-1;
4     vc<ll> z(n,0);
5     forn(i,1,n){
6         if(r>i) z[i]=min(z[i-1],r-i);
7         while(i+z[i]<n && s[i+z[i]]==s[z[i]])
8             ↪ z[i]++;
9         if(i+z[i]>r) r=i+z[i],l=i;
10    }
11    return z;
12 }
13 vc<ll> kmp(const string &s){
14     ll n; n=s.length();
15     vc<ll> k(n,0);
16     forn(i,1,n){
17         ll j; j=k[i-1];
18         while(j>0 && s[i]!=s[j]) j=k[j-1];
19         if(s[i]==s[j]) k[i]=j+1;
20    }
21    return k;
22 }

```